Arithmetic Expression Evaluator

Work in teams of 1-3 (max 3) students | One assignment per group | Due November 10, 2026

The Task

The provided Haskell code implements a simple arithmetic expression evaluator that can be tested using GHCi. It compiles and works on the "happy path", but it includes numerous flaws and bugs. The code is provided separately in a single file called Evaluator.hs (make sure to download the code along with this assignment file). Use Haskell GHC to implement this assignment.

Investigate behaviour before changing the implementation. From the directory containing Evaluator.hs:

```
ghci Evaluator.hs
```

Inside GHCi, try:

```
evaluate "2 + 3 * 4"
evaluate "(2 + 3) * 4"
evaluate "-3 * (4 + 5.5)"
```

Part 1: Debug & Refactor

Debug and refactor this Haskell code:

1. Identify and fix at least **five (5) distinct edge cases** that give incorrect answers or errors.
2. Improve the code structure by organizing it into multiple logically separated modules.

Part 2: Extend

- Add integer floor division (//) and modulo (%) operations.
- Ensure both operators have the same precedence as multiplication (*) and division (/). Handle negative operands and division-by-zero cases correctly.
- Examples: -7 // 3 = -3, -7 % 3 = 2.

Deliverables

- **Code**: A zip file containing the fixed and refactored Haskell code.
- **Report**: A 3-5-page PDF report that includes:
    - A short fault summary with explanations of fixes and extensions.
    - A table of five distinct debugging experiments (input, expected result, before fixing, after fixing).
    - Screenshots of actual GHCi/terminal experiment runs.
    - Declaration of AI-use (tools and scope).